Python小练习：卷积神经网络(CNN)和ResNet18中间特征层可视化

作者：凯鲁嘎吉 - 博客园 http://www.cnblogs.com/kailugaji/

1．卷积神经网络(CNN)中间特征层可视化

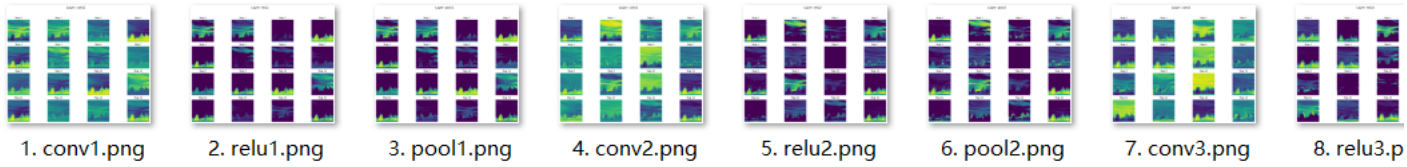
写一个简单的Python程序，实现一个简单的CNN模型，并提供了可视化中间层特征图的功能。主要特点包括：使用hook机制捕获中间层输出、对特征图进行智能可视化处理（处理全相同值的情况）、有序保存各层特征图、完整的图像预处理流程。

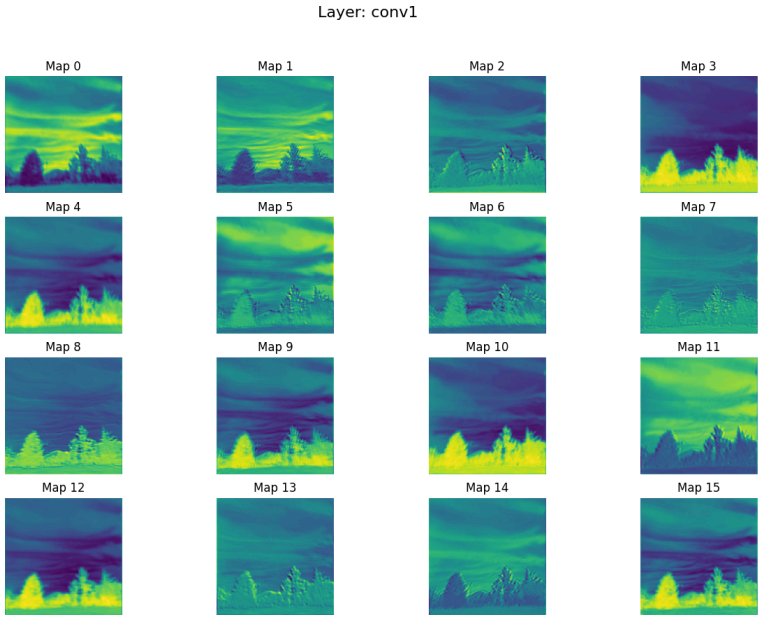
```
# 这段代码实现了一个简单的CNN模型，并提供了可视化中间层特征图的功能。
# 主要特点包括：
# 使用hook机制捕获中间层输出
# 对特征图进行智能可视化处理（处理全相同值的情况）
# 有序保存各层特征图
# 完整的图像预处理流程
# 作者：凯鲁嘎吉 - 博客园 http://www.cnblogs.com/kailugaji/

import torch
import torch.nn as nn
import torchvision.transforms as transforms
from torchvision.utils import save_image
import os
from PIL import Image
import matplotlib.pyplot as plt
import numpy as np
import random

# 固定所有随机种子，确保可复现性
def set_seed(seed=42):
    torch.manual_seed(seed)       # PyTorch CPU随机种子
    torch.cuda.manual_seed(seed)  # PyTorch GPU随机种子
    torch.cuda.manual_seed_all(seed)  # 如果使用多GPU
    np.random.seed(seed)          # NumPy随机种子
    random.seed(seed)             # Python随机种子
    torch.backends.cudnn.deterministic = True  # 确保CUDA卷积运算确定性
    torch.backends.cudnn.benchmark = False     # 关闭cuDNN自动优化

set_seed(42)  # 设置随机种子为42（可改为任意整数）

# 定义一个简单的CNN模型
class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()

        # 网络层定义
        # conv1: 第一卷积层，输入通道3(RGB)，输出通道16，3x3卷积核，步长1，填充1
        # relu1: ReLU激活函数
        # pool1: 最大池化层，2x2池化窗口，步长2
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, stride=1, padding=1)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)

        # conv2: 第二卷积层，输入通道16，输出通道32
        # relu2: ReLU激活函数
        # pool2: 最大池化层
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, stride=1, padding=1)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)

        # conv3: 第三卷积层，输入通道32，输出通道64
        # relu3: ReLU激活函数
        self.conv3 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1)
        self.relu3 = nn.ReLU()

        # 用于存储中间层输出的hook 创建一个空字典来存储中间层的特征图
        self.feature_maps = {}

    # 为每一层注册前向hook，用于捕获该层的输出
    def forward(self, x):
        # 注册hook来捕获中间层输出
        hooks = []
        for name, layer in self.named_children():
            hook = layer.register_forward_hook( # register_forward_hook会在每次前向传播后调用指定的函数
                lambda module, input, output, name=name: self.feature_maps.__setitem__(name, output)
            )
            # 使用lambda函数将每层的输出存储到feature_maps字典中，以层名作为键
            hooks.append(hook)

        # 前向传播
        # 标准的CNN前向传播流程：卷积→激活→池化→卷积→激活→池化→卷积→激活
        x = self.conv1(x)
        x = self.relu1(x)
        x = self.pool1(x)

        x = self.conv2(x)
        x = self.relu2(x)
        x = self.pool2(x)

        x = self.conv3(x)
        x = self.relu3(x)

        # 移除hook 移除所有hook，避免内存泄漏
        for hook in hooks:
            hook.remove()

        return x
```

```
    # 提供获取特征图的方法
    def get_feature_maps(self):
        return self.feature_maps


# 定义图像预处理流程：
# 调整大小为128x128
# 转换为PyTorch张量
# 使用ImageNet数据集的均值和标准差进行归一化
def preprocess_image(image_path):
    transform = transforms.Compose([
        transforms.Resize((128, 128)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ])
    image = Image.open(image_path).convert('RGB')
    image = transform(image).unsqueeze(0)  # 添加batch维度 unsqueeze(0)添加batch维度(从C×H×W变为1×C×H×W)

    return image


# 可视化并保存特征图（修改后的版本）
def visualize_feature_maps(feature_maps, save_dir='feature_maps'):
    os.makedirs(save_dir, exist_ok=True)  # 确保目录存在

    # 添加层序号计数器
    layer_idx = 1

    # 遍历每层的特征图，获取每层的特征图数量
    for layer_name, maps in feature_maps.items():
        # 获取特征图数量
        num_maps = maps.size(1)

        # 为每个特征图创建子图
        plt.figure(figsize=(15, 10))
        plt.suptitle(f'Layer: {layer_name}', fontsize=16)

        # 创建大图，设置标题
        # 限制最多显示16个特征图，避免太多
        display_num = min(16, num_maps)

        # 创建4x4的子图网格
        # 获取第i个特征图，并转换为numpy数组
        for i in range(display_num):
            plt.subplot(4, 4, i + 1)
            # 获取单个特征图并转为numpy
            img = maps[0, i].detach().cpu().numpy()

            # 如果特征图所有值相同，显示为灰度图并标注"Constant"
            # 否则归一化到[0,1]范围并显示为彩色图
            # 检查特征图是否为全零或所有值相同
            if np.all(img == img[0, 0]):  # 如果所有值相同
                plt.imshow(img, cmap='gray')
                plt.title(f'Map {i} (Constant)')
            else:
                # 只有最大值不等于最小值时才进行归一化
                if img.max() != img.min():
                    img = (img - img.min()) / (img.max() - img.min())
                plt.imshow(img, cmap='viridis')
                plt.title(f'Map {i}')

            plt.axis('off')

        # 保存图像，文件名前加上序号（如 "1. conv1.png"）
        save_path = os.path.join(save_dir, f'{layer_idx}. {layer_name}.png')
        plt.savefig(save_path, bbox_inches='tight')
        plt.close()

        print(f'Saved feature maps for {layer_name} to {save_path}')
        layer_idx += 1  # 更新序号


# 主函数
def main():
    # 初始化模型
    model = CNN()

    # 加载并预处理图像
    image_path = 'input_image.jpg'  # 替换为你的图像路径
    input_image = preprocess_image(image_path)

    # 前向传播
    _ = model(input_image)

    # 获取特征图
    feature_maps = model.get_feature_maps()

    # 可视化并保存特征图
    visualize_feature_maps(feature_maps)

if __name__ == '__main__':
    main()
```

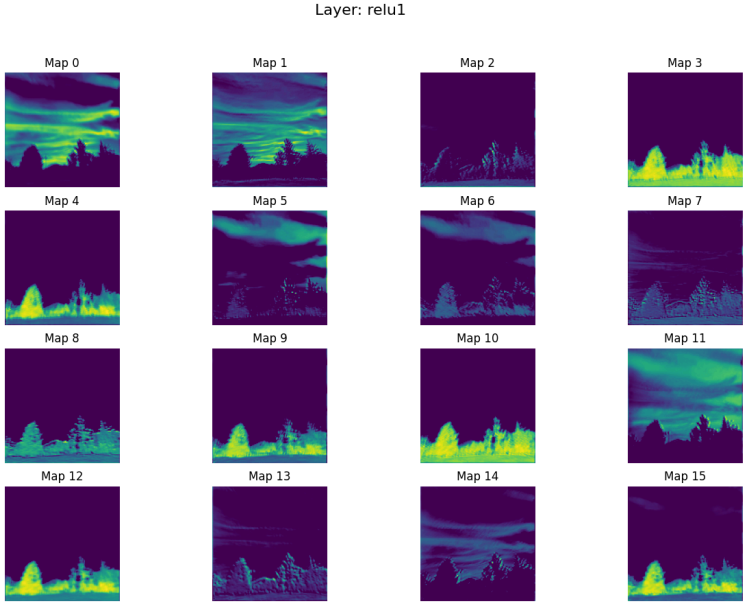
输入图像：

输出结果：



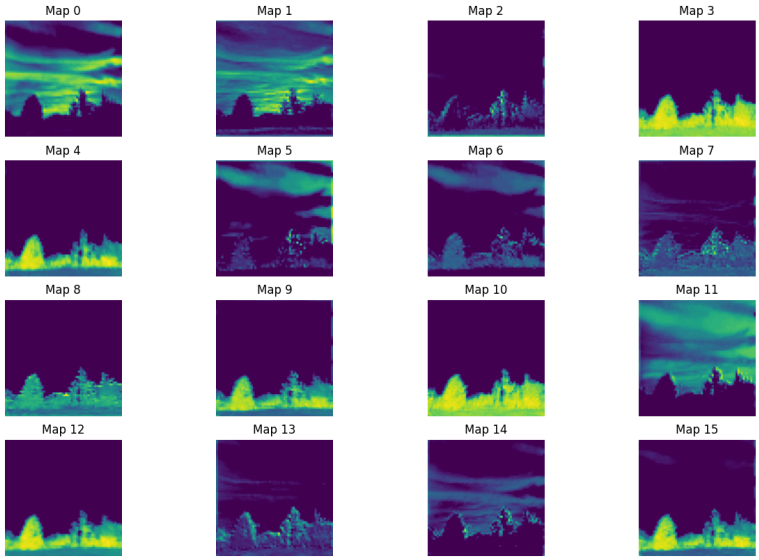


1. conv1.png



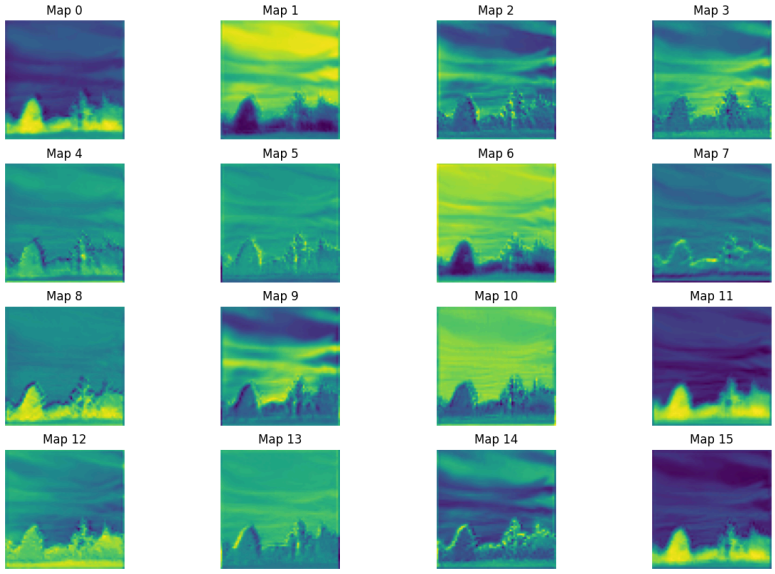
2. relu1.png

Layer: pool1



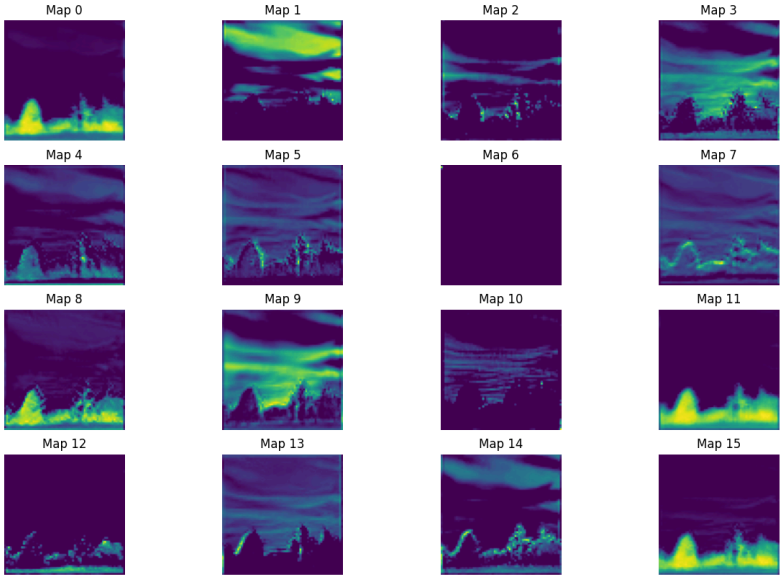
3. pool1.png

Layer: conv2



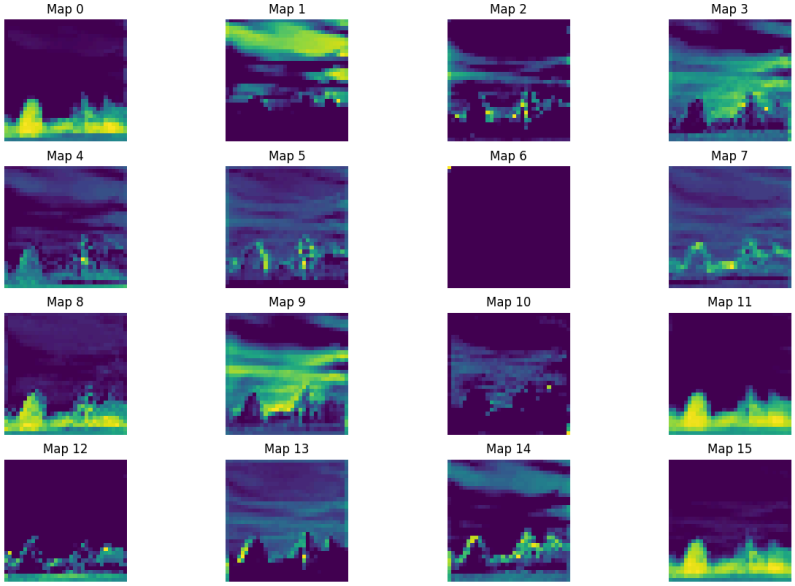
4. conv2.png

Layer: relu2



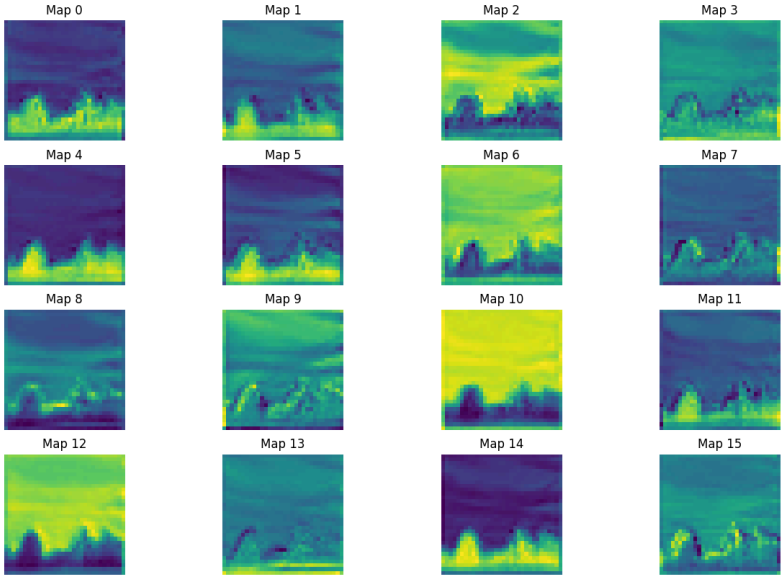
5. relu2.png

Layer: pool2



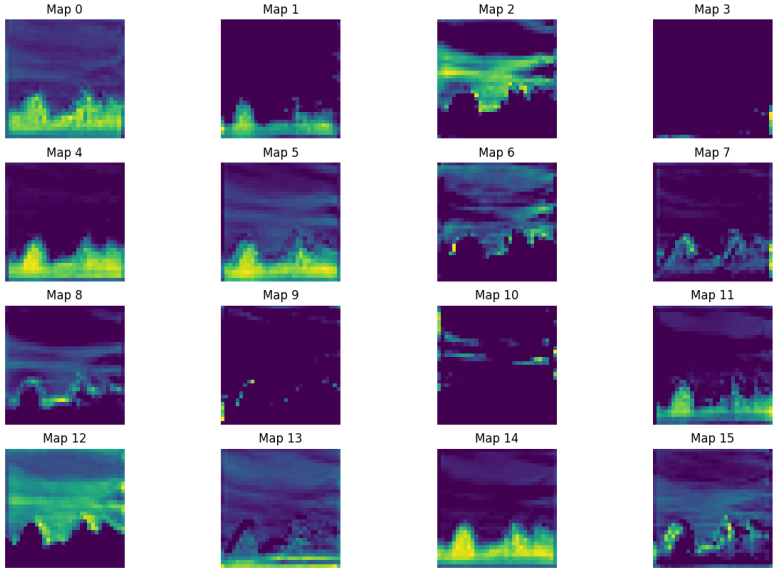
6. pool2.png

Layer: conv3



7. conv3.png

Layer: relu3



8. relu3.png

2．ResNet18中间特征层可视化

写一个简单的Python程序，加载预训练的ResNet18模型、对输入图像进行前向传播、捕获所有中间层的输出并将这些输出可视化为特征图保存到本地目录。

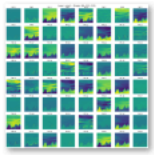
```
# 加载预训练的ResNet18模型
# 对输入图像进行前向传播
# 捕获所有中间层的输出
# 并将这些输出可视化为特征图保存到本地目录
# 作者：凯鲁嘎吉 - 博客园 http://www.cnblogs.com/kailugaji/

import torch
import torch.nn as nn
from torchvision import models, transforms
from PIL import Image
import matplotlib.pyplot as plt
import numpy as np
import os

# 创建名为'layer_outputs'的目录用于保存输出结果
os.makedirs('layer_outputs', exist_ok=True)

# 定义hook函数来获取中间层输出
# 这个类用于捕获神经网络的中间层输出
class LayerActivations:
    def __init__(self):
        self.activations = None

    def __call__(self, module, input, output):
        self.activations = output.detach()  # 确保不追踪梯度


# 加载预训练模型(这里使用ResNet18作为示例)
model = models.resnet18(pretrained=True)
model.eval()  # 将模型设为评估模式(关闭dropout等训练专用层)

# 定义图像预处理
preprocess = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
# 定义图像预处理流程:
# 调整大小为256x256
# 中心裁剪为224x224
# 转换为张量
# 使用ImageNet的均值和标准差进行归一化

# 加载输入图像
image_path = 'input_image.jpg'  # 替换为你的图片路径
image = Image.open(image_path)
input_tensor = preprocess(image)
input_batch = input_tensor.unsqueeze(0)  # 创建batch维度 unsqueeze(0)添加batch维度(从3D变为4D张量)

# 注册hook来获取所有中间层
activation_records = {}
hooks = []

# 遍历模型的所有层并注册hook
# 遍历模型的所有层
# 对于Sequential容器(如ResNet的layer1-4)，进一步遍历其子层
# 为每个层注册hook，将输出保存到activation_records字典中

# 遍历模型的所有直接子层(named_children返回层名称和层对象)
# 对于ResNet18来说，这些子层包括：conv1, bn1, relu, maxpool, layer1, layer2, layer3, layer4, avgpool, fc
for name, layer in model.named_children():
    # 检查当前层是否是nn.Sequential容器
    # 在ResNet中，layer1-layer4都是Sequential容器，包含多个残差块
    if isinstance(layer, nn.Sequential):
        # 如果是Sequential容器，需要进一步遍历其内部的子层
        for sub_name, sub_layer in layer.named_children():
            # 创建完整的层名称，格式为"外层名_内层名"(如"layer1_0")
            full_name = f"{name}_{sub_name}"
            # 为该子层创建一个LayerActivations实例，用于捕获其输出
            activation_records[full_name] = LayerActivations()

            # 为该子层注册forward hook，并将hook句柄存入hooks列表
            # 当该层前向传播时，会自动调用LayerActivations的__call__方法保存输出
            hooks.append(sub_layer.register_forward_hook(activation_records[full_name]))
    else:
        # 对于非Sequential的单个层(如conv1, bn1等)
        # 直接为该层创建LayerActivations实例
        activation_records[name] = LayerActivations()
        # 注册forward hook
        hooks.append(layer.register_forward_hook(activation_records[name]))

# 前向传播
# 禁用梯度计算以节省内存
# 执行前向传播，这会触发所有注册的hook
with torch.no_grad():
    output = model(input_batch)

# 移除所有hook，防止内存泄漏
for hook in hooks:
    hook.remove()


# 可视化每一层的输出
# 可视化2D特征图(通道、高度、宽度)
# 最多显示64个特征图
```

```
# 8列网格布局
# 每个特征图单独归一化显示
# 保存为PNG文件
def visualize_layer_output(layer_name, activations, input_image, index):
    # 获取该层的输出
    activations = activations.activations.squeeze(0)  # 去除batch维度

    # 检查激活的形状
    if activations.dim() == 1:
        print(f"跳过 {layer_name} - 一维输出不适合可视化")
        return

    # 对于2D特征图 (C, H, W)
    if activations.dim() == 3:
        num_feature_maps = min(activations.size(0), 64)

        # 计算网格大小
        cols = 8
        rows = (num_feature_maps + cols - 1) // cols

        # 创建子图
        fig, axes = plt.subplots(nrows=rows, ncols=cols, figsize=(16, 2 * rows))
        fig.suptitle(f'Layer: {layer_name} - Shape: {tuple(activations.shape)}', fontsize=16)

        for i, ax in enumerate(np.array(axes).flat):
            if i < num_feature_maps:
                # 获取单个特征图并归一化
                feature_map = activations[i].numpy()
                feature_map = (feature_map - feature_map.min()) / (feature_map.max() - feature_map.min() + 1e-8)

                # 显示特征图
                ax.imshow(feature_map, cmap='viridis')
                ax.set_title(f'FM {i}')
            ax.axis('off')

        plt.tight_layout()
        plt.savefig(f'layer_outputs/{index:1d}. {layer_name}.png', bbox_inches='tight')
        plt.close()
    else:
        print(f"跳过 {layer_name} - 不支持的维度 {activations.dim()}D")


# 处理并保存每一层的输出
# 遍历所有记录的层输出
# 调用可视化函数保存结果
for idx, (layer_name, record) in enumerate(activation_records.items(), start=1):
    visualize_layer_output(layer_name, record, image, idx)

# 保存原始输入图像作为参考
plt.figure(figsize=(8, 8))
plt.imshow(np.array(image))
plt.title('Original Image')
plt.axis('off')
plt.savefig('layer_outputs/0. original_image.png', bbox_inches='tight')
plt.close()

print(f"所有中间层输出已保存到 layer_outputs 目录")
```

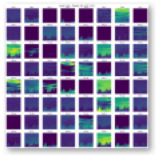
输入图像：

输出结果：

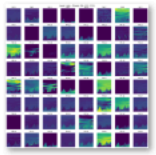
0. original_image.png



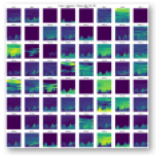
1. conv1.png



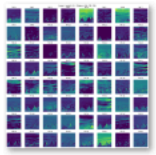
2. bn1.png



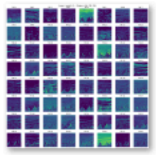
3. relu.png



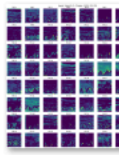
4. maxpool.png



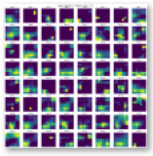
5. layer1_0.png



6. layer1_1.png

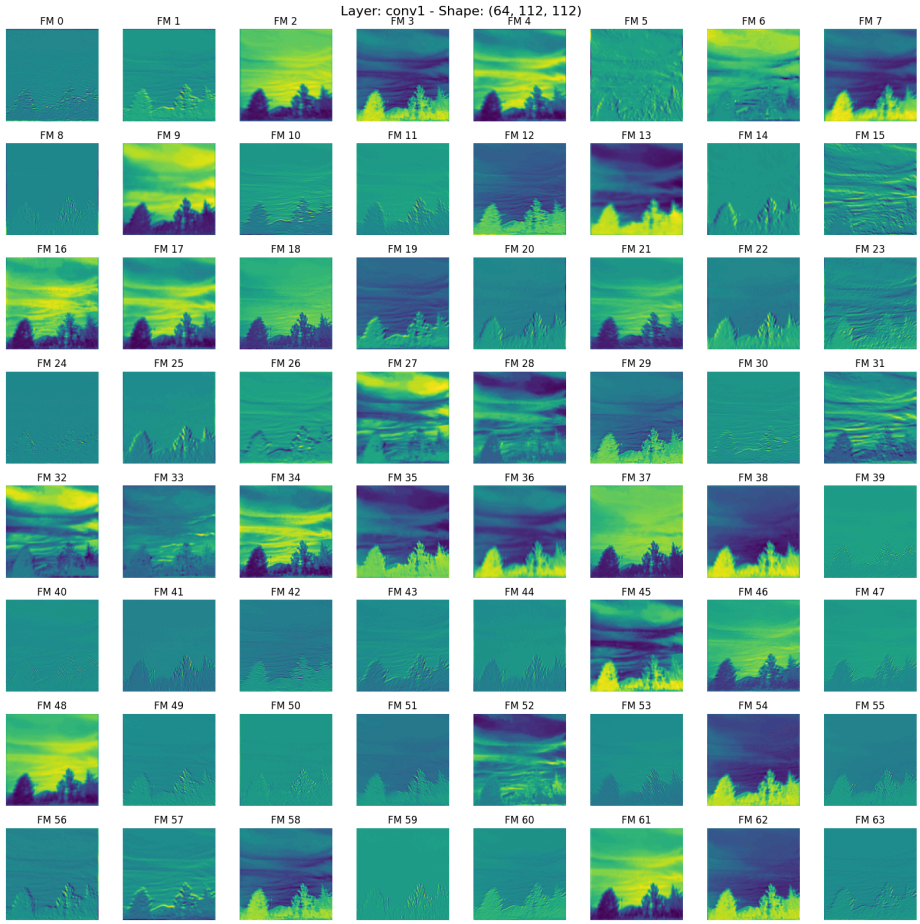


7. layer2_0.

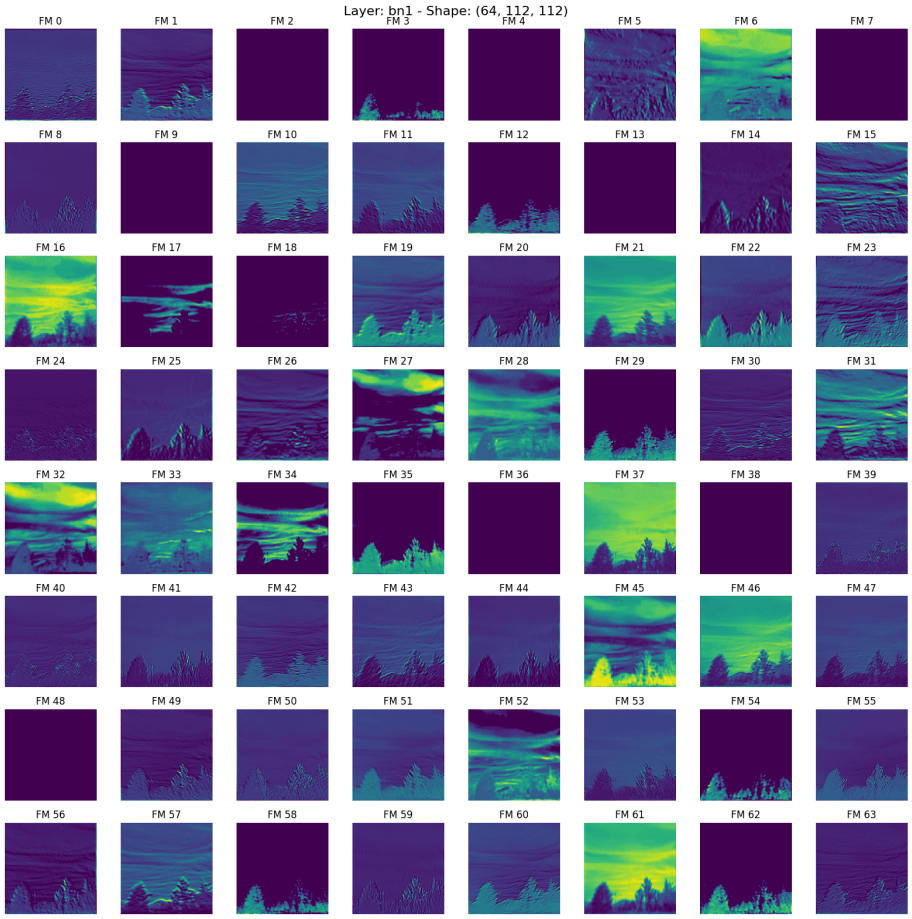


12. layer4_1.png

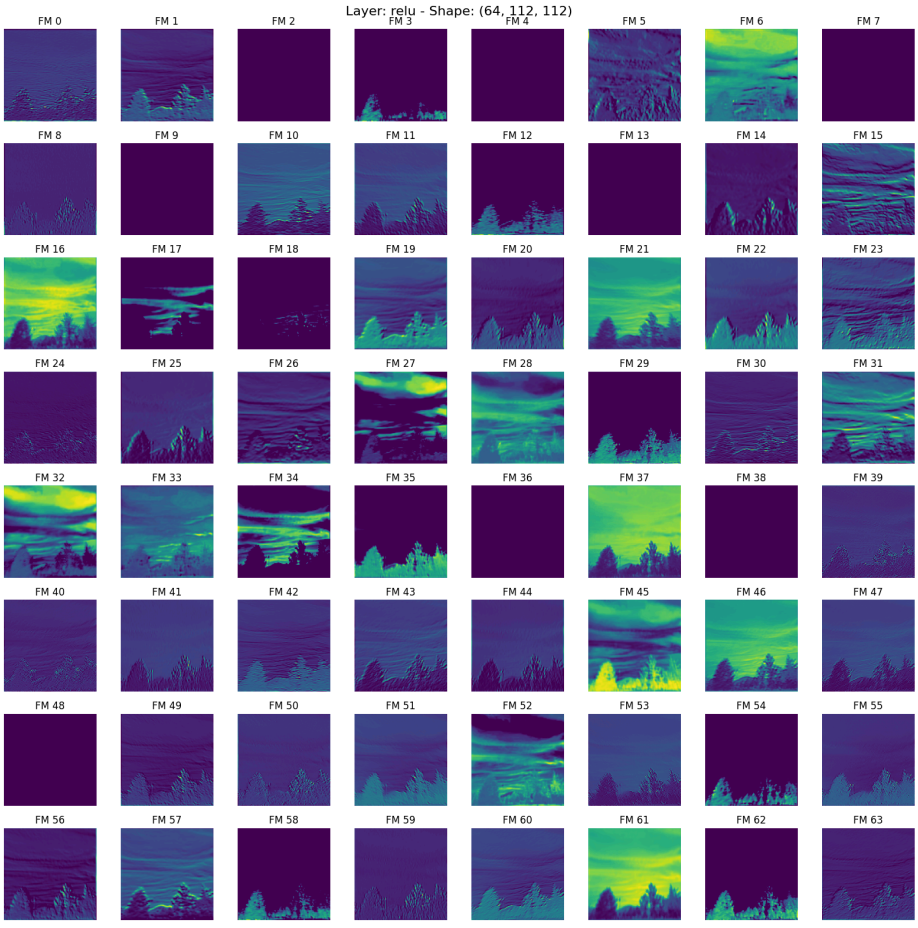
13. avgpool.png



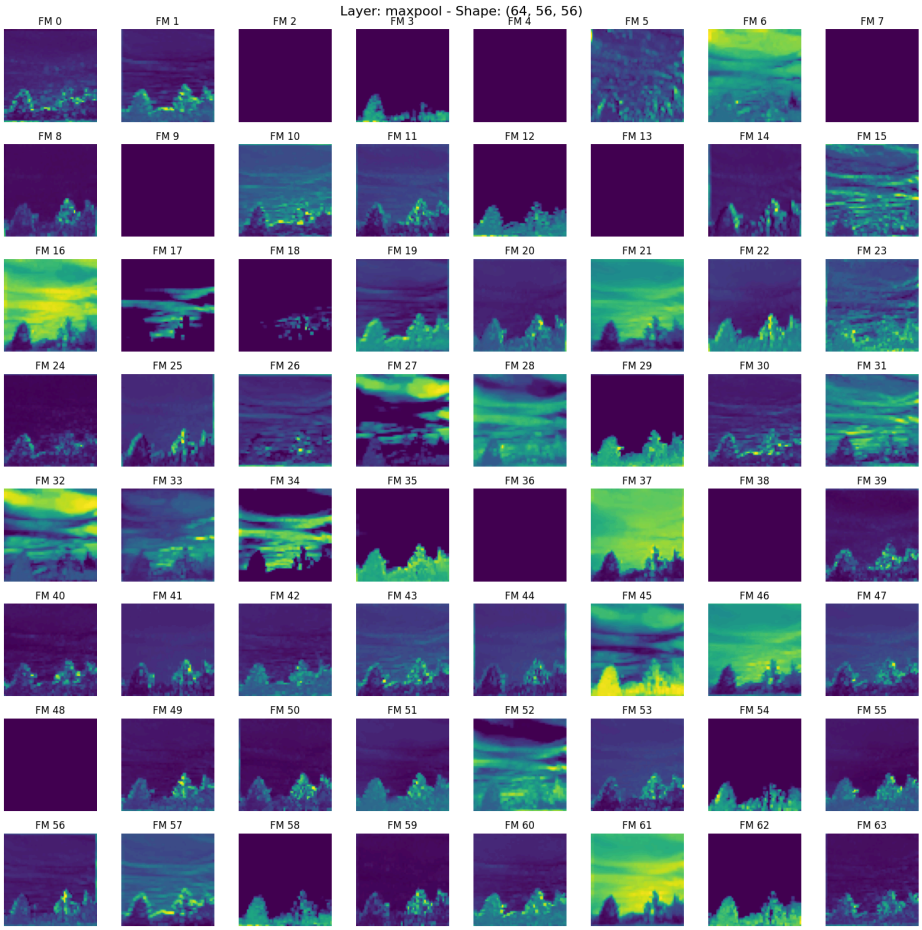
1. conv1.png



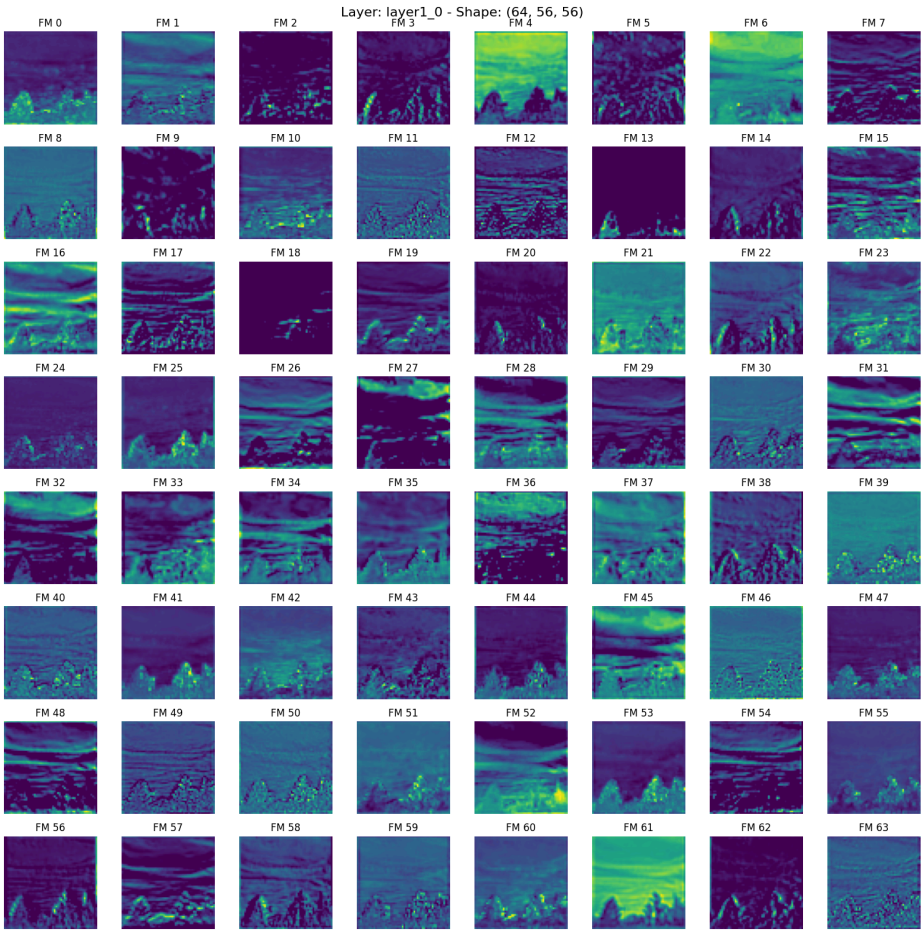
2. bn1.png



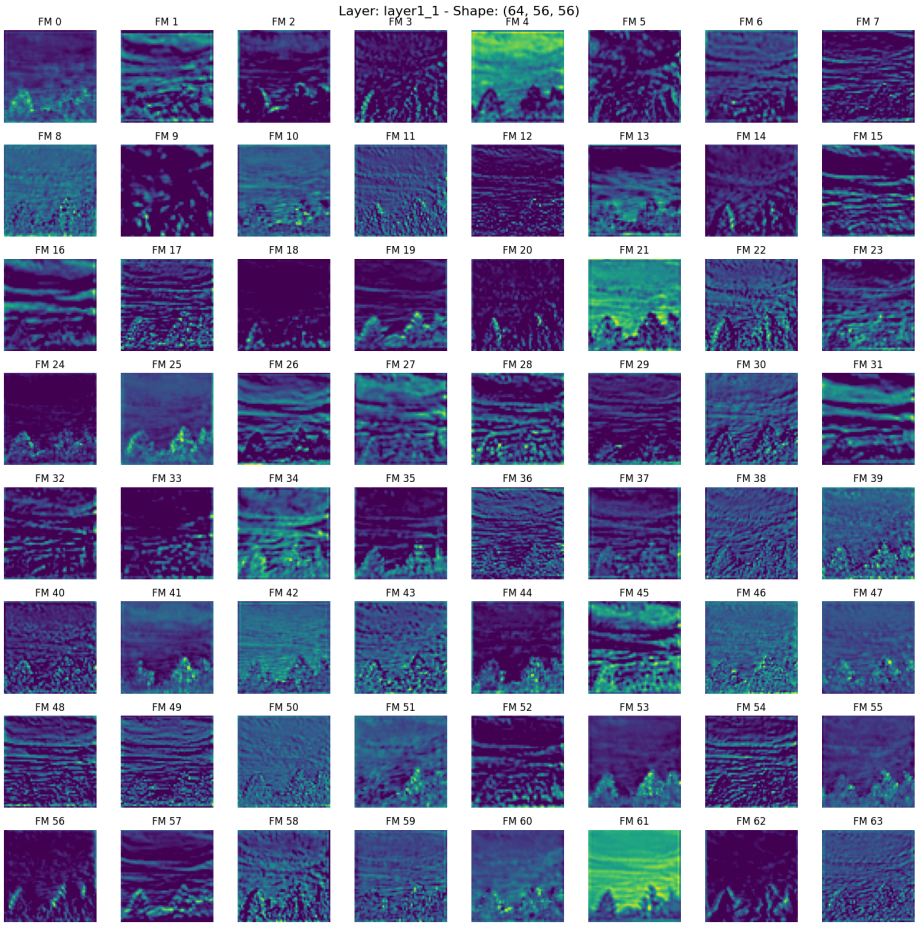
3. relu.png



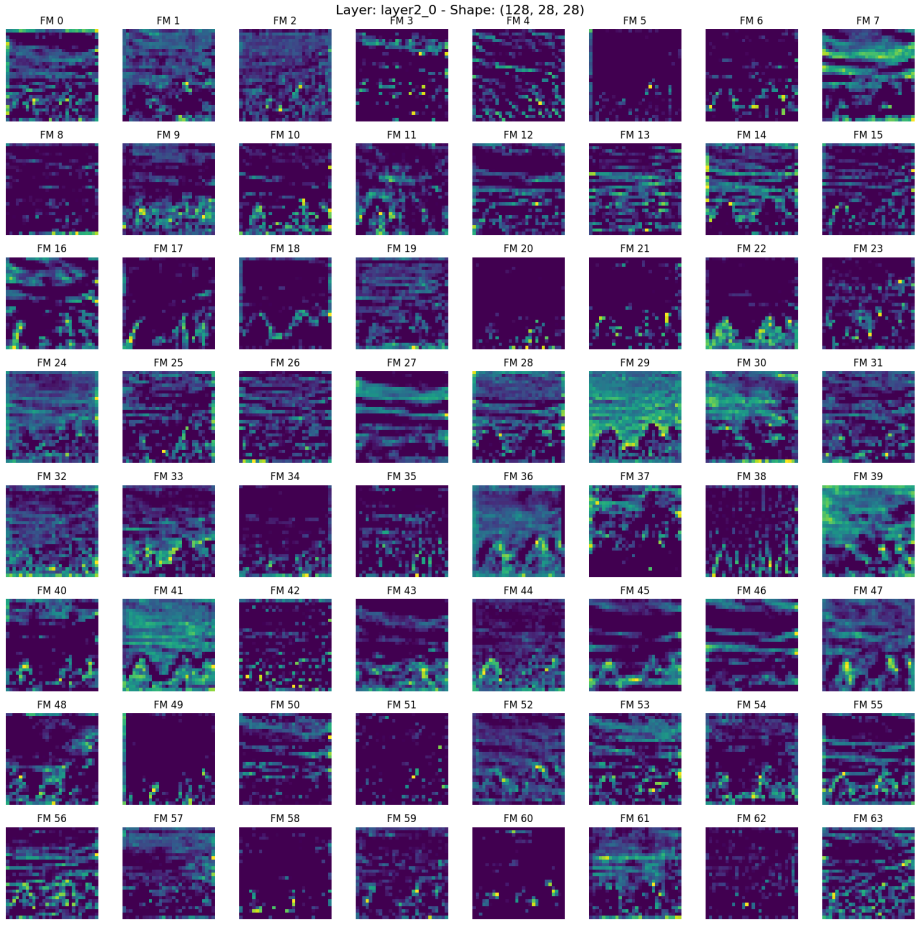
4. maxpool.png



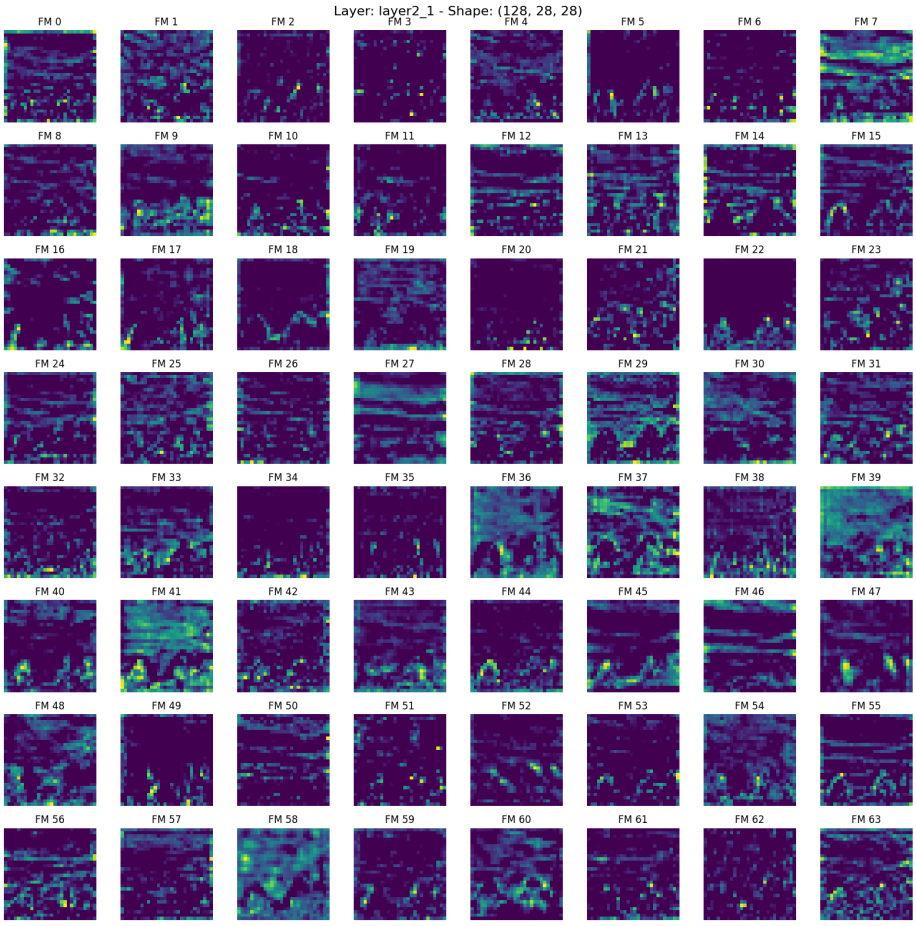
5. layer1_0.png



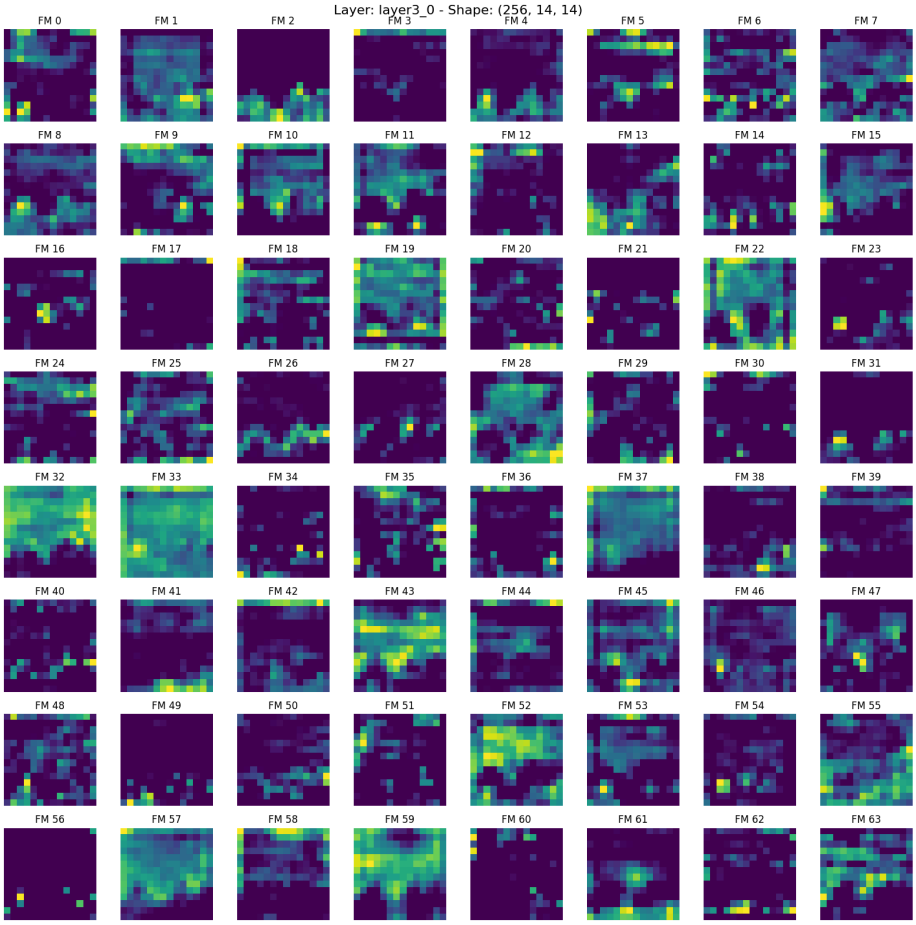
6. layer1_1.png



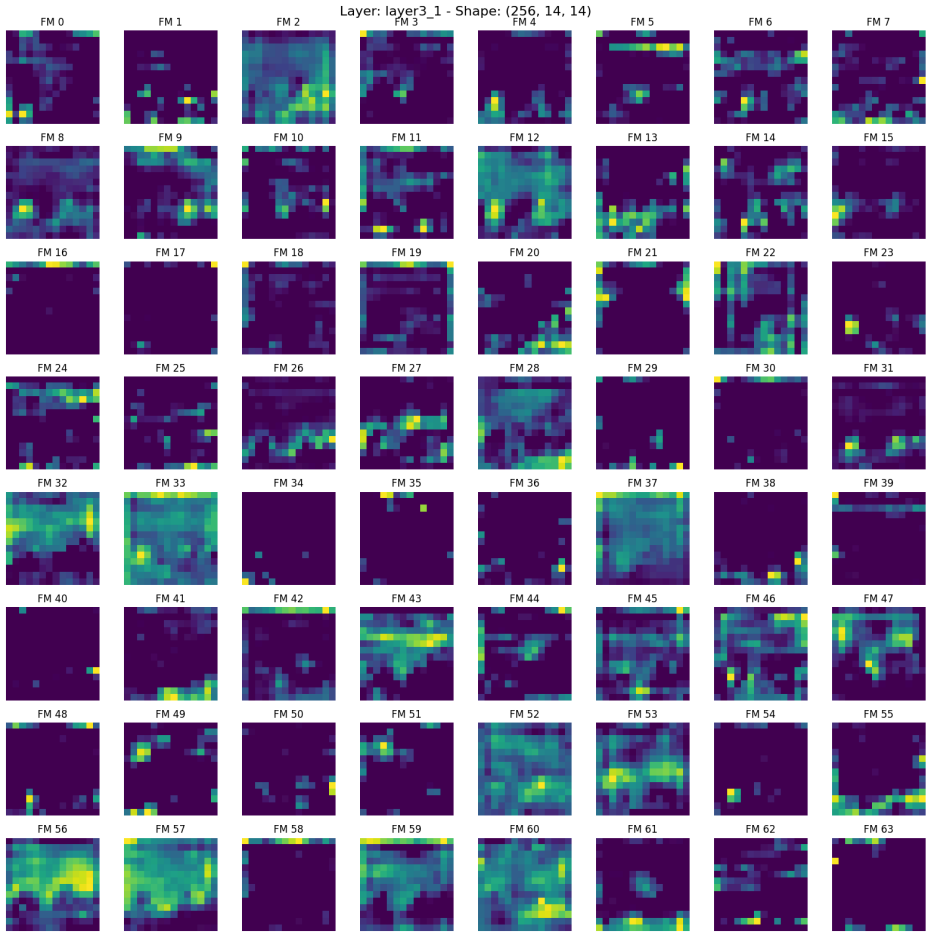
7. layer2_0.png



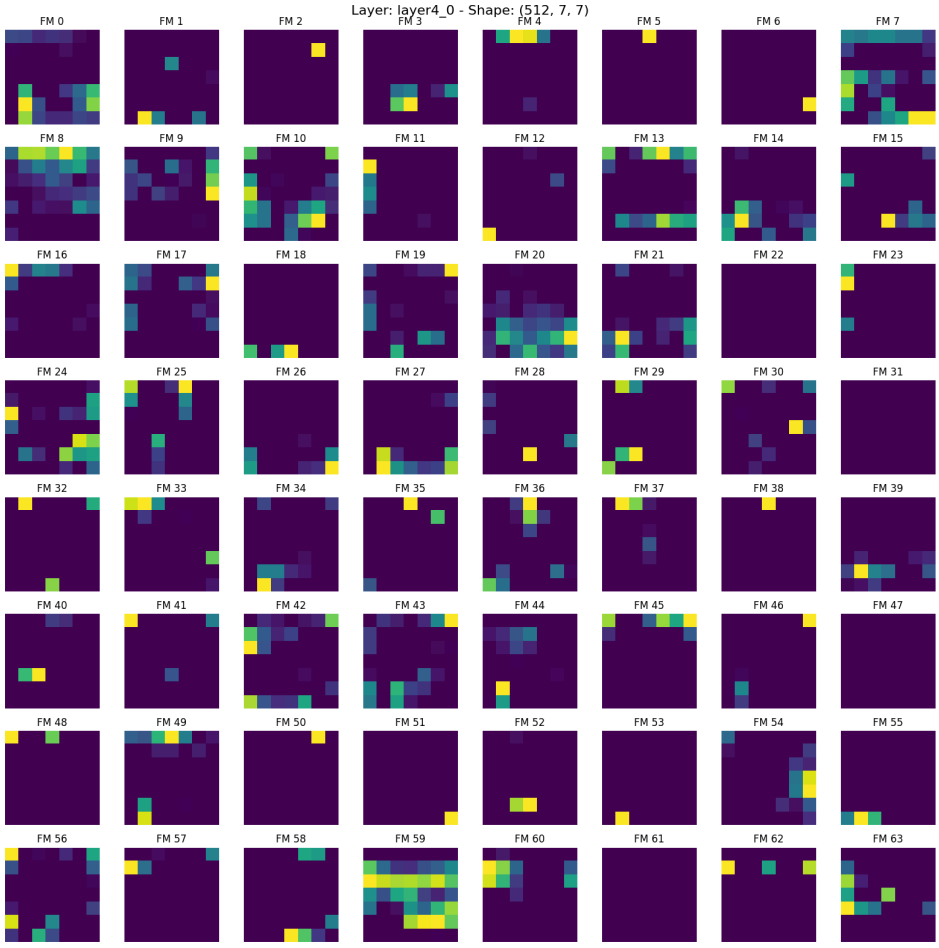
8. layer2_1.png



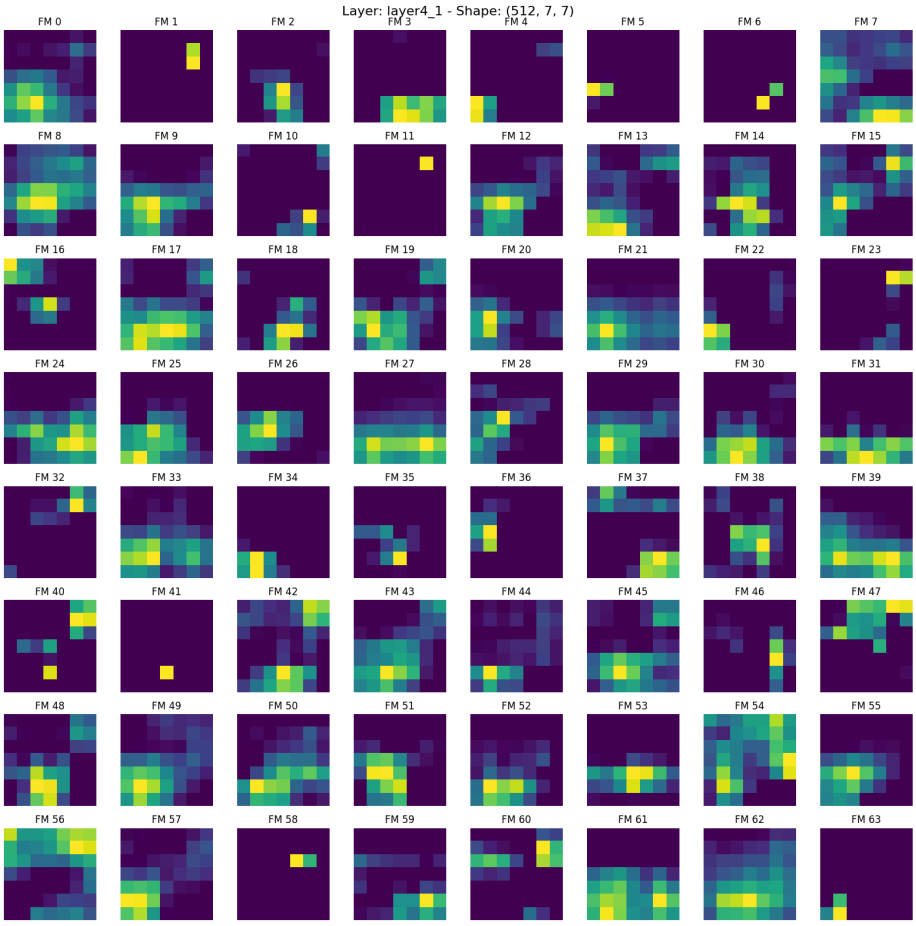
9. layer3_0.png



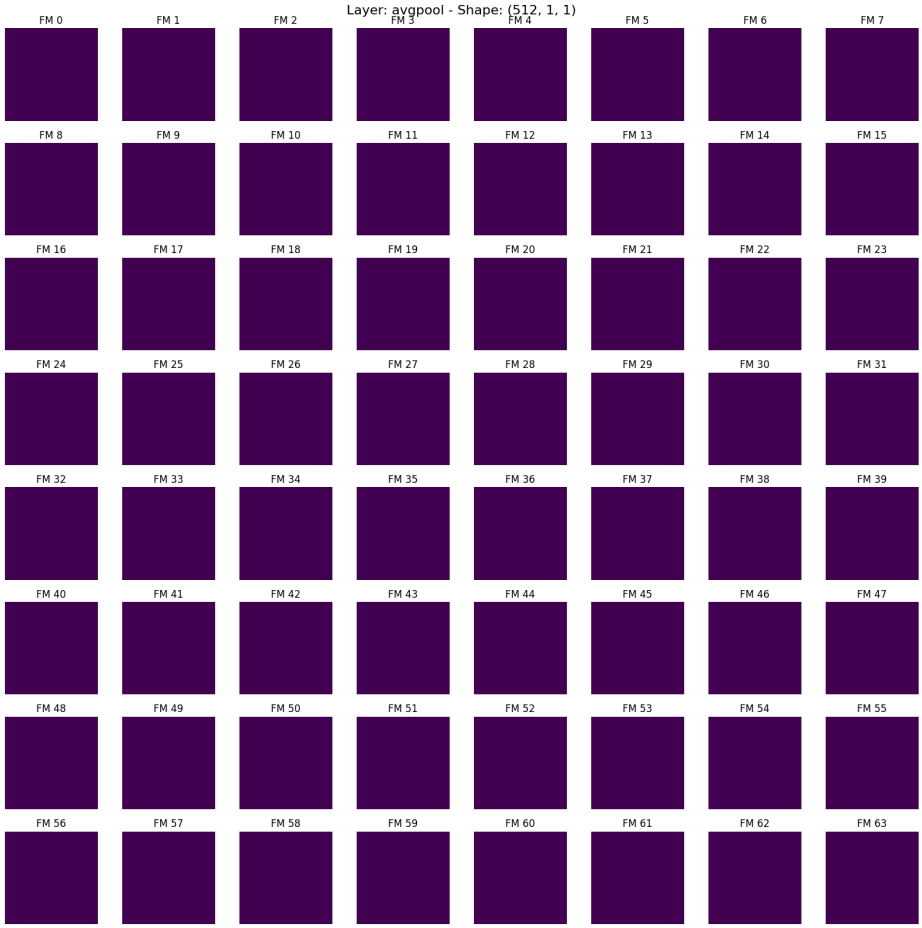
10. layer3_1.png



11. layer4_0.png



12. layer4_1.png



13. avgpool.png